AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 20.10.2026

Lab 03

Instrukcje warunkowe

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć, **jak program podejmuje decyzje**: logika Boole'a w praktyce
- Operatory porównania i logiczne: `== != < > <= >=`, `and`, `or`, `not`
- Rozgałęzienia `if` / `elif` / `else` i wyrażenie warunkowe
- Zbudować prosty **system decyzyjny** „Jak zainwestować?"
- Efekty z sylabusa: **U1**, **U3** (algorytmy decyzyjne), **KS1**



CZĘŚĆ I

Teoria: logika w programie

Typ `bool` i prawdziwość

- `bool` ma dwie wartości: `True` i `False` (podtyp `int`: `True == 1`)
- **Każdy obiekt** ma wartość logiczną – to ona decyduje w `if`

FAŁSZYWE (*FALSY*)	PRAWDZIWE (*TRUTHY*)
`False`, `None`	`True`
`0`, `0.0`	każda liczba ≠ 0
`""` (pusty tekst)	`"0"`, `" "`, `"False"`
`[]`, `()`, `{}`, `set()`	niepuste kolekcje

```
name = input("Imię: ")
if not name:           # pusty tekst jest fałszywy
    print("Nie podano imienia")
```

Operatory porównania

OPERATOR	ZNACZENIE	PRZYKŁAD
`==`	równe	`2 + 2 == 4` → `True`
`!=`	różne	`"a" != "A"` → `True`
`<`, `>`, `<=`, `>=`	mniejsze, większe…	`3 <= 3` → `True`

- Porównania można **łączyć łańcuchowo**, jak w matematyce:

```
18.5 <= bmi < 25      # to samo co: 18.5 <= bmi and bmi < 25
```

- Teksty porównujemy **leksykograficznie** (wg kodów Unicode): `"Zebra" < "apple"` → `True`
- `==` porównuje **wartości**, `is` – **tożsamość** obiektów; `is` używamy tylko z `None`

Operatory logiczne

A	B	A AND B	A OR B	NOT A
`True`	`True`	`True`	`True`	`False`
`True`	`False`	`False`	`True`	`False`
`False`	`True`	`False`	`True`	`True`
`False`	`False`	`False`	`False`	`True`

- **Priorytet:** `not` > `and` > `or` – w razie wątpliwości nawiasy
- **Prawa De Morgana:**
  - `not (a and b)` == `(not a) or (not b)`
  - `not (a or b)` == `(not a) and (not b)`

Leniwe wartościowanie (*short-circuit*)

- `and` przestaje liczyć przy pierwszym fałszu, `or` – przy pierwszej prawdzie
- Dzięki temu warunek może się **zabezpieczać sam**:

```
b = 0
if b != 0 and 10 / b > 1:   # 10 / b nie zostanie obliczone
    print("duży iloraz")
```

- `and` i `or` zwracają **jeden z argumentów**, nie zawsze `bool`:

```
"" or "domyślne"     # 'domyślne'
"Anna" or "domyślne" # 'Anna'
0 and 1 / 0          # 0 – dzielenie się nie wykona
```

`if` / `elif` / `else`

```
if bmi < 18.5:
    category = "niedowaga"
elif bmi < 25:
    category = "waga prawidłowa"
elif bmi < 30:
    category = "nadwaga"
else:
    category = "otyłość"
```

- **Wcięcie wyznacza blok** (PEP 8: 4 spacje). Błąd wcięcia → `IndentationError`
- Warunki sprawdzane są **od góry**; wykona się tylko **pierwszy** prawdziwy blok
- Dlatego w `elif bmi < 25` nie trzeba pisać `bmi >= 18.5` – to już wiadomo

Kolejność warunków ma znaczenie

```python
# ŹLE – drugi warunek nigdy nie zadziała
if points > 60:
    grade = 3
elif points > 91:
    grade = 5
```

- Zasada: od warunku **najbardziej szczegółowego** do **najogólniejszego**
- Każdą gałąź sprawdź **wartościami granicznymi**: 60, 61, 91, 92
- Testy do `zad4_ocena.py` robią dokładnie to

Wyrażenie warunkowe i `match`

- **Wyrażenie warunkowe** (operator trójargumentowy, PEP 308):

```
status = "pełnoletni" if age >= 18 else "niepełnoletni"
```

- **`match` / `case`** (Python 3.10, PEP 634) – dopasowanie wzorców:

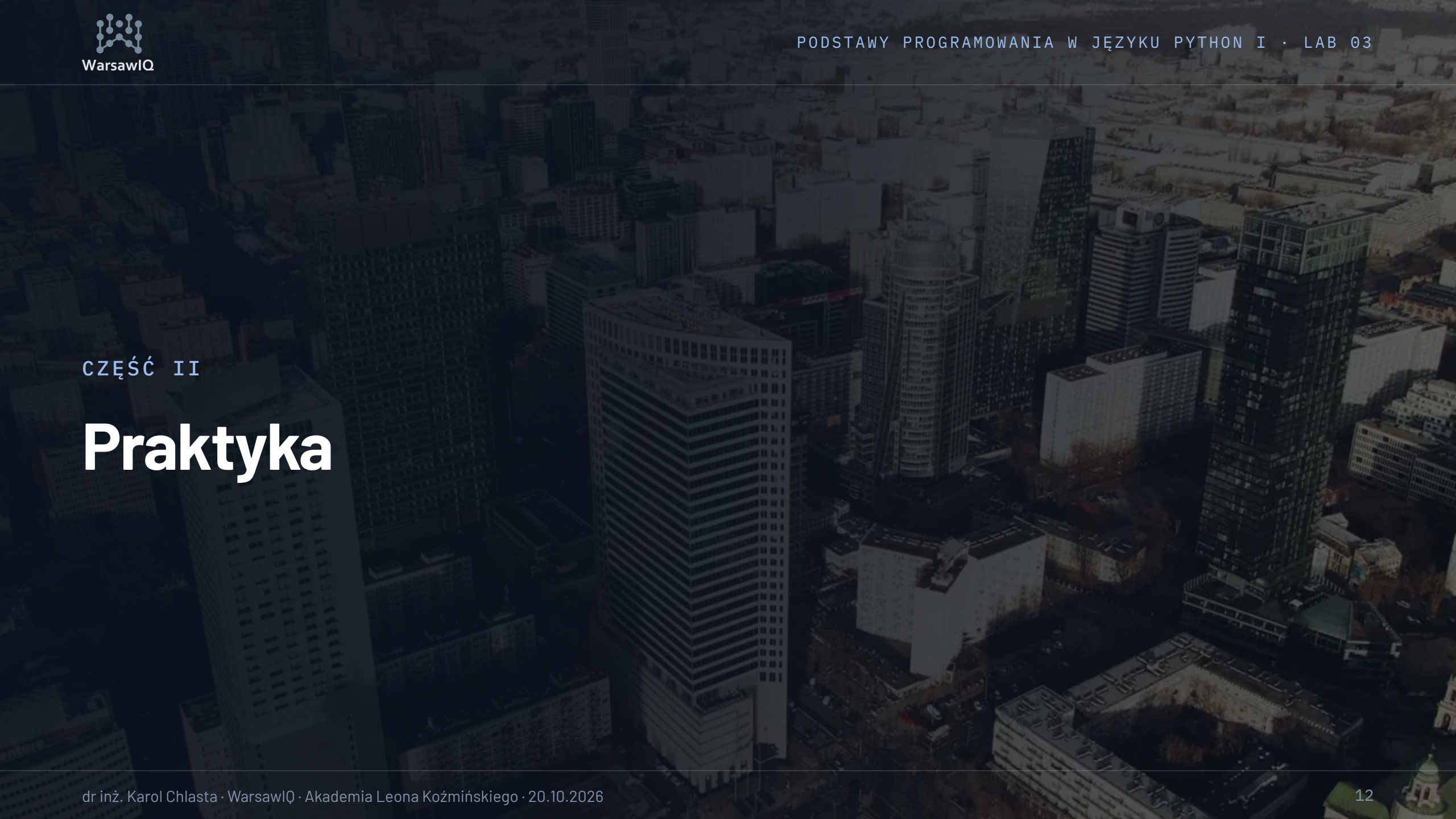
```
match op:
    case "+":
        result = a + b
    case _:
        print("Nieznany operator")
```

- Na tych zajęciach wystarczy `if` / `elif` – `match` to ciekawostka na przyszłość

Algorytm decyzyjny: tablica reguł

#	WARUNEK	REKOMENDACJA
1	kwota < 1000 zł	poduszka finansowa
2	horyzont < 2 lata	lokata
3	ryzyko niskie	obligacje skarbowe
4	ryzyko średnie	fundusz mieszany
5	ryzyko wysokie, horyzont ≥ 5	fundusz akcji (ETF)
6	ryzyko wysokie, horyzont < 5	fundusz mieszany
7	cokolwiek innego	nieznany profil

- Tablica reguł przekłada się **1:1** na łańcuch `if` / `elif`
- Tak działają proste **systemy ekspertowe** i reguły biznesowe w bankach



CZĘŚĆ II

Praktyka

Dane od użytkownika są „brudne”

```
risk = input("Profil ryzyka: ").strip().lower()
```

WPISANO	PO `.STRIP().LOWER()`
`"Niskie"`	`"niskie"`
`" NISKIE "`	`"niskie"`
`"średnie"`	`"średnie"`

- `strip()` usuwa białe znaki z brzegów, `lower()` zamienia na małe litery
- **Normalizuj dane, zanim je porównasz**

Ciekawostki: logika i kalendarz 📅

- **George Boole** opisał algebrę logiki w *An Investigation of the Laws of Thought* (1854) – od jego nazwiska pochodzi typ `bool`
- Regułę „co 4 lata, ale nie co 100, chyba że co 400" wprowadził **kalendarz gregoriański** (1582); Polska przyjęła go jako jeden z pierwszych krajów
- **Excel** do dziś traktuje 1900 jako rok przestępny – celowo, dla zgodności z arkuszem Lotus 1-2-3
- **Microsoft Zune** (30 GB) zawiesiły się masowo 31.12.2008 – pętla w sterowniku zegara źle obsługiwała ostatni dzień roku przestępnego

Dzisiejsze zadania

	PLIK	ZADANIE
★	`zad1_inwestycja.py`	„Jak zainwestować?” – tablica reguł
★	`zad2_kalkulator.py`	kalkulator z wyborem działania
★	`zad3_przestepny.py`	rok przestępny
★	`zad4_ocena.py`	ocena wg progów z sylabusa
☆	`zad5_trojkat.py`	rodzaj trójkąta

Sprawdzenie i oddanie

```
cd lab03_warunki
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 03"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- Po zajęciach – **0 pkt**, bez wyjątków
- Pamiętaj: **Sync fork** na początku zajęć, żeby pobrać nowy lab

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 5
- M. Dawson, *Python dla każdego*, Helion 2015 – rozdz. 3
- Dokumentacja: *Truth Value Testing* i *Boolean Operations* (*docs.python.org/3/library/stdtypes.html*)
- PEP 634 – *Structural Pattern Matching*

Następny lab (27.10): pętle